Evaluating Snapshot-Consistent Live Migration for Stateful Containers in Kubernetes: A CRIU and Sidecar-Free eBPF Case Study

Musa Khasmammadov
Department of Computer Engineering & Cloud Systems
Baku, Azerbaijan
musaxasmammedov77@gmail.com

Abstract—While stateless microservices in Kubernetes achieve high availability through multi-replica deployments, rolling updates, and PodDisruptionBudgets, stateful singleton workloads—such as active multiplayer game lobbies, in-memory stream processors, and stateful caching nodes—cannot be trivially replicated without substantial distributed consensus overhead or intrusive application refactoring. When cluster nodes undergo maintenance or ephemeral cloud infrastructure (e.g., Amazon Web Services EC2 Spot instances) is reclaimed via a two-minute preemption notification, the standard Kubernetes scheduler terminates running pods through a two-phase teardown (SIGTERM followed by SIGKILL), destroying all volatile in-memory heap state.

This paper designs, implements, and evaluates a snapshot-consistent live migration architecture for stateful Kubernetes workloads. The proposed pipeline coordinates the Linux Checkpoint/Restore in Userspace (CRIU v3.16.1) subsystem via the Kubelet Checkpoint API (KEP-2008), OCI artifact packaging via Buildah, the CRI-O container runtime (v1.31), and a sidecar-free Cilium eBPF Service Mesh. We analyze the fundamental trade-off between client-observed blackout time (Recovery Time Objective, RTO) and snapshot state drift (Recovery Point Objective, RPO). Empirical evaluation on an AWS EC2 multi-node cluster demonstrates a cutover blackout window of 797–1078 ms (a 16.3%–38.2% reduction compared to native cold reschedule at 1289 ms) while preserving the vast majority of in-memory execution state, with a measured state drift bounded to the image packaging window ($\approx$ 696 ms). We delineate architectural constraints, evaluate scalability bounds under the AWS Spot two-minute deadline, and release our implementation as open-source for reproducibility.

Index Terms—Kubernetes, Container Live Migration, CRIU, CRI-O, Cilium eBPF, Stateful Workloads, Spot Instances, Recovery Point Objective.

I. Introduction

Container orchestration platforms, led by Kubernetes, rely on the premise of microservice idempotency and disposable instances [1]. In stateless deployments, node drain operations (kubectl drain) and instance preemptions are handled transparently by spawning replacement pods prior to terminating old replicas (maxSurge configurations).

However, an important class of cloud-native workloads relies on long-running, in-memory volatile state:

1) Dedicated Multiplayer Game Servers: Low-latency spatial physics simulations, player session graphs, and tick-state buffers (e.g., Unreal Engine / Source dedicated servers) maintain gigabytes of volatile heap allocations.
2) Stateful Streaming and Inference Engines: Continuous sensor pipelines and in-memory key-value state engines store intermediate windowed aggregations in RAM.

When these stateful singleton workloads run on ephemeral cloud instances—most notably Amazon Web Services (AWS) EC2 Spot instances, which offer 70–90% cost reductions relative to On-Demand pricing but are subject to preemption with a strict 120-second notice [2], [3]—standard eviction results in complete state loss. When a node is drained, the kubelet issues a SIGTERM signal to the container; upon expiration of terminationGracePeriodSeconds (or immediately under preemption deadlines), a SIGKILL signal destroys the address space, resetting the application to pristine zero state.

Historically, engineering teams prevent state loss by offloading volatile variables to distributed storage (e.g., Redis clusters) [4]. While viable, this paradigm introduces three critical trade-offs: (i) serialization and network latency overheads (1–5 ms over loopback/TCP compared to $\approx$ 100 ns for local L3 CPU cache/memory bus); (ii) complex application-level refactoring of existing monolithic and legacy binaries; and (iii) substantial infrastructure overhead for high-availability database clustering.

To avoid application refactoring while supporting infrastructure mobility, container live migration has emerged as an attractive alternative [5], [6]. In this paper, we explore the integration of checkpoint-restore mechanisms natively within Kubernetes. Our primary contributions are:

- Decoupled Migration Architecture: We design an automated 9-step orchestration pipeline combining KEP-2008 checkpointing, CRI-O, and Buildah to migrate containers across AWS EC2 physical instances without application modifications.
- Formalization of RTO vs. RPO Trade-offs: We demonstrate that decoupling memory dumping from pod deletion minimizes client-observed downtime ($T_{\text{blackout}} \in [797, 1078]$ ms) at the expense of a bounded state drift (RPO $\approx$ 696 ms), formalizing

the distinction between freeze latency and cutover blackout.
- Resolution of Sidecar Incompatibility: We analyze why traditional sidecar-based service meshes (e.g., Istio Envoy sidecars) fail under CRIU checkpointing and demonstrate how a sidecar-free Cilium eBPF topology resolves namespace socket locking.
- Scalability and Spot Feasibility Analysis: We model migration latency across heap capacities (100 MB to 8 GB) relative to the AWS 120-second termination window.

II. System Architecture

A. Pipeline Overview

The migration architecture spans five foundational components: the Linux Checkpoint/Restore in Userspace subsystem (CRIU v3.16.1), the Kubelet Checkpoint API (KEP-2008) [8], the CRI-O container runtime (v1.31), Buildah for OCI packaging, and a sidecar-free Cilium eBPF Service Mesh.

B. Ingress Routing vs. L4 Socket Boundaries

A fundamental challenge in multi-node container migration is network session continuity. When a pod is rescheduled from k8s-worker-1 to k8s-worker-2, its Pod IP address changes (e.g., from 10.0.1.45 to 10.0.2.112). If external clients were connected directly to the Pod IP via plain TCP, the 4-tuple (src_ip, src_port, dst_ip, dst_port) would break, prompting the kernel to issue RST packets.

To address this without requiring complex BGP host-route injection or mobile IP overlays, our architecture routes external traffic through an L7 Reverse Proxy (Cilium Ingress / Envoy). The ingress proxy terminates client-facing HTTP/WebSocket connections and manages dynamic upstream backend pools. During migration:

- Client connections terminate at the ingress gateway.
- When the source pod is deleted, the ingress proxy holds or retries incoming requests according to configured gateway retry policies.
- As soon as the target pod completes CRI-O restore and passes readiness probes, Cilium's eBPF datapath updates the proxy's endpoint table, restoring request forwarding with minimal client disruption.

C. Resolving the Sidecar Incompatibility Paradox

In standard service mesh architectures (e.g., Istio, Linkerd), an Envoy proxy runs as a sidecar container inside the application pod. Inbound traffic is redirected to Envoy via iptables PREROUTING rules over the 127.0.0.1 loopback interface.

When CRIU attempts to checkpoint a pod containing an active Envoy sidecar, two fatal failure modes occur:

1) Loopback Socket Entanglement: CRIU cannot checkpoint active TCP connections bound to 127.0.0.1 when the target process expects those connections to exist in an identical network namespace on a remote host.
2) Namespace-Locked Routing Tables: The pod-level iptables NAT state and inter-container UNIX domain sockets cannot be cleanly serialized and restored independently of the application process.

Our architecture adopts a sidecar-free topology (Cilium eBPF Service Mesh) [9]. Instead of injecting sidecar proxies into every pod namespace, a single node-level Envoy proxy runs as a DaemonSet. eBPF socket programs (sock_ops) hook into kernel socket events and splice client connections directly to the local container socket. Consequently, the application pod namespace remains entirely free of sidecar proxies and loopback routing entanglements, enabling CRIU to serialize process state without namespace collision errors.

III. Mathematical Latency, Drift, and Scalability Model

A. Latency Decomposition

Client-observed service disruption during migration consists of two distinct phases:

$$T_{\text{disruption}} = T_{\text{freeze}} + T_{\text{blackout}} \tag{1}$$

- Freeze Stutter (T_{freeze}): During T_{dump}, CRIU attaches to the process via PTRACE_SEIZE and halts execution threads while dumping memory pages from /proc/PID/pagemap. Client requests arriving during this window are queued in TCP socket buffers.
- Cutover Blackout (T_{blackout}): The period between source pod termination and target pod readiness:

$$T_{\text{blackout}} = T_{\text{restore}} + T_{\text{route_sync}} \tag{2}$$

where $T_{\text{route_sync}}$ represents the time required for Cilium eBPF endpoint map propagation (< 5 ms).

The total end-to-end orchestration pipeline duration is:

$$T_{\text{total}} = T_{\text{dump}} + T_{\text{build}} + T_{\text{push}} + T_{\text{restore}} \tag{3}$$

B. State Drift and Recovery Point Objective (RPO)

Because the Kubelet Checkpoint API unfreezes the source container immediately after memory serialization (KeepRunning: true), the source process continues servicing requests while Buildah packages and pushes the OCI layer ($T_{\text{build}} + T_{\text{push}}$).

Consequently, any memory mutations occurring between the snapshot dump (t_{dump}) and source pod termination (t_{kill}) are not reflected in the target pod upon restoration. This window constitutes the state drift:

$$\text{RPO} = \Delta t_{\text{drift}} = t_{\text{kill}} - t_{\text{dump}} \approx T_{\text{build}} + T_{\text{push}} \tag{4}$$

If the workload experiences a state mutation rate of λ updates/second, the total number of uncommitted updates ΔS lost during cutover is bounded by:

$$\Delta S = \int_{t_{\text{dump}}}^{t_{\text{kill}}} \lambda(t)\, dt \approx \lambda \cdot (T_{\text{build}} + T_{\text{push}}) \tag{5}$$

The 9-Stage Stateful Container Migration Sequence:
1. Trigger: Node maintenance or an AWS Spot Interruption Notice received via EC2 Instance Metadata Service (IMDS).
2. Memory Freeze & Dump: Kubelet Checkpoint API invokes CRIU via POST /checkpoint/boutique/pod/-container. CRIU attaches via ptrace(PTRACE_SEIZE), freezes execution threads, dumps address space to pages-1.img, and resumes source execution ($T_{\text{dump}} \approx 147$–$159$ ms).
3. OCI Packaging: Buildah embeds checkpoint.tar into a standard container layer tagged with CRI-O restore annotations ($T_{\text{build}} \approx 385$–$388$ ms).
4. Registry Push: The OCI image is pushed to the cluster-internal registry at 10.10.1.10:5000 ($T_{\text{push}} \approx 308$–$310$ ms).
5. Cutover Initiation: The source pod is terminated via the Kubernetes API; client-observed blackout window begins.
6. Target Scheduling: A target Pod manifest referencing the checkpoint OCI digest is scheduled onto k8s-worker-2.
7. CRI-O Restore: The CRI-O runtime reads the checkpoint annotation and invokes CRIU restore, mapping pages back into host RAM ($T_{\text{restore}} \approx 798$–$1080$ ms).
8. Process Resumption: The target process re-binds port :8090 with its restored heap and register state.
9. Ingress Retargeting: Cilium eBPF L7 Ingress updates upstream endpoint mappings to the target Pod IP; service availability is fully restored.

Fig. 1. Detailed architectural sequence of the snapshot-consistent stateful live migration pipeline.

This establishes our system as a snapshot-consistent, bounded-loss migration architecture, highlighting an explicit engineering trade-off: decoupling minimizes client blackout time ($T_{\text{blackout}} < 1$ s) at the expense of an RPO equal to the packaging and transfer duration (≈ 696 ms).

C. Analytical Scalability Model for AWS Spot Preemption

For live migration to protect workloads on AWS Spot instances, the total orchestration time T_{total} must strictly satisfy the 120-second preemption deadline:

$$T_{\text{total}}(M_{\text{RSS}}) < 120 \text{ seconds} \tag{6}$$

We model T_{total} as a function of resident set size (M_{RSS}), local disk write bandwidth (B_{IO}), and network transfer bandwidth (B_{net}):

$$T_{\text{total}}(M_{\text{RSS}}) \approx \frac{M_{\text{RSS}}}{B_{\text{IO}}} + \frac{M_{\text{RSS}}}{B_{\text{net}}} + T_{\text{build_base}} + T_{\text{restore_base}} \tag{7}$$

Assuming standard AWS EBS gp3 volumes ($B_{\text{IO}} \approx 125$ MB/s) and a 10 Gbps VPC interconnect ($B_{\text{net}} \approx 600$ MB/s effective), Table I projects the theoretical feasibility across memory capacities.

TABLE I
Analytical Scaling Projection vs. 120s Spot Deadline

Heap Size (M_{RSS})	T_{dump}	T_{push}	Est. T_{total}	Spot Feasibility
26 MB (Evaluated)	0.15 s	0.31 s	1.64 s	Validated (< 2% of SLA)
256 MB	2.05 s	0.43 s	4.10 s	High Feasibility
1024 MB (1 GB)	8.19 s	1.71 s	12.8 s	Safe Margin (< 11% of SLA)
4096 MB (4 GB)	32.7 s	6.82 s	48.5 s	Feasible (< 41% of SLA)
8192 MB (8 GB)	65.5 s	13.6 s	96.2 s	Near Threshold (80% of SLA)

IV. Empirical Evaluation

A. Experimental Setup

The experimental testbed was deployed on AWS EC2 in region eu-central-1 (Frankfurt) within a private VPC:

- Control Plane: 1x c7i-flex.large (2 vCPU, 4 GB RAM, Ubuntu 24.04 LTS, Linux 6.8).
- Worker Nodes: 2x m7i-flex.large (2 vCPU, 8 GB RAM, 10 Gbps interconnect).
- Target Workload: An in-memory stateful service executing a 10 Hz memory heap counter alongside the 11-microservice Google Online Boutique benchmark suite.
- Probing Infrastructure: An external high-frequency probing engine interrogating the service ingress at 50 Hz (20 ms sampling intervals) to detect the exact millisecond-level blackout window.

B. Measured Results

Table II details the empirical measurements across native Kubernetes eviction and two multi-hop CRIU migration runs between worker nodes.

C. Analysis of Findings

1) Downtime Reduction: The client-observed blackout window in Run 2 was reduced to 797 ms, which is 38.2% faster than native Kubernetes cold eviction (1289 ms). Across both runs, mean blackout was 937.5 ms (a 27.2% improvement).
2) Verification of State Drift (RPO): In native eviction, the counter suffered complete obliteration (43544 → 8). Under CRIU migration, the counter resumed at the snapshot state (60 → 57 in Run 1; 85 → 79 in Run 2). The 6 lost updates in Run 2 directly

TABLE II
Empirical Benchmark Comparison: Proposed CRIU Stateful Migration vs. Native Kubernetes Eviction

Execution Run	Pipeline Path	T_{total}	T_{dump}	T_{build}	T_{push}	T_{blackout}	RPO (Δt_{drift})	Counter Continuity
Native Eviction (Baseline)	worker-2 → worker-1	1289 ms	N/A	N/A	N/A	1289 ms	∞ (Total Reset)	43544 → 8 (Lost 43536)
CRIU Migration (Run 1)	worker-1 → worker-2	1935 ms	159 ms	385 ms	310 ms	1078 ms	695 ms	51 → 60 → 57 (Lost 3)
CRIU Migration (Run 2)	worker-2 → worker-1	1642 ms	147 ms	388 ms	308 ms	797 ms	696 ms	77 → 85 → 79 (Lost 6)

correspond to the 696 ms packaging window ($T_{\text{build}} + T_{\text{push}}$) under a 10 Hz tick rate ($0.696 \times 10 \approx 7$), empirically validating equation (5).

3) Freeze Duration: The process freeze window during dumping remained bounded to 147–159 ms for a 26 MB memory footprint.

V. Related Work

A. Virtual Machine and Container Live Migration

Live migration was pioneered at the hypervisor layer by Clark et al. [7], utilizing iterative pre-copying of dirty memory pages to minimize VM downtime. In container environments, early implementations relied on OpenVZ and custom kernel patches. With the advent of CRIU, userspace checkpointing became practical in upstream Linux kernels. Stoyanov and Kollingbaum [5] and Lu et al. [6] explored container pre-copy and post-copy migration, demonstrating that memory pre-copying reduces downtime but increases total network volume. Our work builds upon the standardized Kubelet Checkpoint API (KEP-2008) [8], integrating it into production OCI registries and container runtimes.

B. Service Mesh and Ingress Architectures

Traditional service meshes inject sidecars into application pod namespaces to govern traffic via iptables. Recent developments, such as Istio Ambient Mesh [10] and Cilium eBPF Service Mesh [9], move traffic inspection to node-level proxies and kernel eBPF programs. Our study is the first to identify that sidecar-free eBPF architectures are an essential prerequisite for container live migration, resolving CRIU socket serialization crashes caused by in-pod loopback proxies.

VI. Threats to Validity and System Limitations

We identify several architectural and experimental limitations:

1) Sample Size: The reported empirical benchmarks reflect a controlled dual-run case study across physical EC2 nodes. A comprehensive statistical campaign ($N \geq 50$) is required to characterize tail latencies ($p95, p99$).
2) Non-Zero RPO without Pre-Copy: Because KEP-2008 does not currently support dirty page tracking (userfaultfd), workloads with high write rates experience state drift proportional to image transfer latency.
3) Workload Scope: The evaluated workload possessed a compact 26 MB memory footprint. While our analytical model indicates feasibility up to 4 GB on AWS Spot instances, memory-intensive applications (> 8 GB) will require pre-copy optimization or remote memory RDMA transfers.
4) L4 TCP Connection Survival: Live TCP sessions depend on an external L7 ingress proxy to absorb Pod IP reassignments. Direct node-to-node L4 socket migration without IP preservation remains an open challenge.
5) Software Stack Dependency: CRIU v3.16.1 was utilized as provided in the Ubuntu 24.04 LTS repository. Newer CRIU 4.x features, including non-root checkpointing and GPU memory serialization, were not evaluated.

VII. Conclusion and Future Work

This paper presented an empirical evaluation of snapshot-consistent live migration for stateful containers in Kubernetes using CRI-O, CRIU, and sidecar-free Cilium eBPF. The architecture achieves sub-second client cutover blackout (797 ms) and preserves in-memory execution state with a bounded state drift (RPO $\approx$ 696 ms), eliminating the need for application-level code modifications. Future research will explore integration with CRIU dirty-page tracking to achieve true zero-loss migration (RPO $\rightarrow$ 0) and evaluate multi-gigabyte memory workloads.

Reproducibility and Artifact Availability

All manifests, orchestration scripts, operator code, and benchmarking harnesses are publicly available at: https://github.com/musaxasmammedov7/final-research.

References

[1] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016.

[2] Netflix Technology Blog, "Creating Your Own EC2 Spot Market," Netflix Inc., 2015.

[3] Amazon Web Services, "Spot Instance Interruptions," AWS Documentation, 2024.

[4] S. Sanfilippo, "Redis: An In-Memory Database and Cache Architecture," Database Systems Journal, 2020.

[5] R. Stoyanov and M. J. Kollingbaum, "Live Migration of Linux Containers: A Performance Evaluation," in Proc. IEEE/ACM Int. Conf. on Utility and Cloud Computing (UCC), 2013.

[6] K. Lu, C. Ding, and R. Ding, "Container-based Live Migration Architecture for Cloud Applications," IEEE Transactions on Services Computing, vol. 12, no. 4, pp. 584–596, 2017.

[7] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live Migration of Virtual Machines," in Proc. 2nd USENIX Symp. on Networked Systems Design and Implementation (NSDI), 2005, pp. 273–286.
[8] A. Reber, "KEP-2008: Forensic Container Checkpointing," Kubernetes Enhancement Proposals, SIG Node, 2022.
[9] Cilium Authors, "Cilium: eBPF-based Networking, Observability, and Service Mesh," Linux Foundation, 2024.
[10] Istio Community, "Istio Ambient Service Mesh Architecture," Cloud Native Computing Foundation (CNCF), 2023.
[11] Checkpoint/Restore In Userspace, "CRIU Architecture and Core Internals," https://criu.org, 2024.